

SmartML Advisor

An Intelligent Machine Learning Algorithm Recommendation System

Submitted by **Abu Bakar Rafi** (ID: 12208039)

Supervisor Md. Zahidur Rahman, Lecturer, Dept. of CSE

Presentation Outline

- 1 Motivation
- 2 Problem Statement
- 3 Aim & Objectives
- 4 Existing System
- 5 Proposed System
- 6 Related Works
- 7 Technologies Used
- 8 System Architecture
- 9 Methodology & Design
- 10 System Development
- 11 Database Design
- 12 Testing & Validation
- 13 Results & Discussion
- 14 Conclusion
- 15 Limitations
- 16 Future Work

Motivation

- Algorithm selection is critical in student projects & early-stage prototypes, yet remains scattered across notebooks, blogs, and ad-hoc peer advice.
- Beginners rarely understand the assumptions behind different ML algorithms.
- Goal: combine automated dataset analysis with transparent, rule-based guidance in one accessible web app.
- Every recommendation is traced back to a human-readable rule — not a hidden statistical score.



Transparent reasoning reduces blind trial-and-error and builds real understanding of algorithm behavior.

— Core motivation behind SmartML Advisor

Problem Statement

- Beginners don't understand the assumptions behind different algorithms.
- Manual trial-and-error doesn't scale as candidate algorithms grow.
- Existing AutoML systems are computationally expensive and act as black boxes.
- Missing values, class imbalance, and problem type are rarely checked before an algorithm is chosen.

Core Problem

No secure, traceable, database-driven system exists that profiles a dataset and explains — in plain language — why one algorithm suits it better than another.

Aim & Objectives

Aim: Build a reliable, database-driven platform that helps users select the most suitable ML algorithm through transparent, rule-based reasoning.

1

Secure multi-user authentication (registration, login, sessions)

2

Automated data profiling of CSV/Excel datasets

3

Rule-based scoring engine to rank ML algorithms

4

Transparent, human-readable justifications

5

Reference library of algorithm strengths & weaknesses

6

Persistent history of uploads, recommendations & feedback

Existing System

Manual Trial-and-Error

Try several algorithms via scikit-learn and compare metrics informally.

- + Flexible, no setup
- Doesn't scale; no explanation

Full AutoML Systems

e.g. Auto-sklearn — automatically searches algorithms & hyperparameters.

- + Highly competitive models
- Expensive & black-box

Meta-Learning Recommenders

Use dataset meta-features to predict the best algorithm.

- + Data-driven accuracy
- Needs large historical meta-dataset

Proposed System — SmartML Advisor

- 1 Secure, authenticated web portal with JWT sessions & bcrypt hashing
- 2 Automated dataset profiling: rows/columns, types, missing values, duplicates, class balance
- 3 Rule-based scoring engine ranking algorithms with a human-readable reason per rule
- 4 Independent algorithm reference library, browsable without any upload
- 5 Persistent upload/recommendation history and user feedback in a normalized MySQL database

Related Works — Feature Gap

Feature	Manual Trial-and-Error	AutoML (Auto-sklearn)	SmartML Advisor
Secure multi-user portal	No	Varies	Yes
Automated dataset profiling	Manual	Yes	Yes
Rule-based explainable scoring	No	No	Yes
Human-readable reasons	No	No	Yes
Algorithm reference library	No	No	Yes
Upload/recommendation history	No	Varies	Yes
Instant result without training	Varies	No	Yes

Technologies Used



FastAPI (Python)

Backend routing, validation, business logic



MySQL + SQLAlchemy

12-table relational schema via ORM



JWT + bcrypt

Session tokens & password hashing



pandas / NumPy / SciPy

Dataset loading & statistical profiling



scikit-learn

Algorithm reference & scoring criteria



Groq API (llama-3.1-8b-instant)

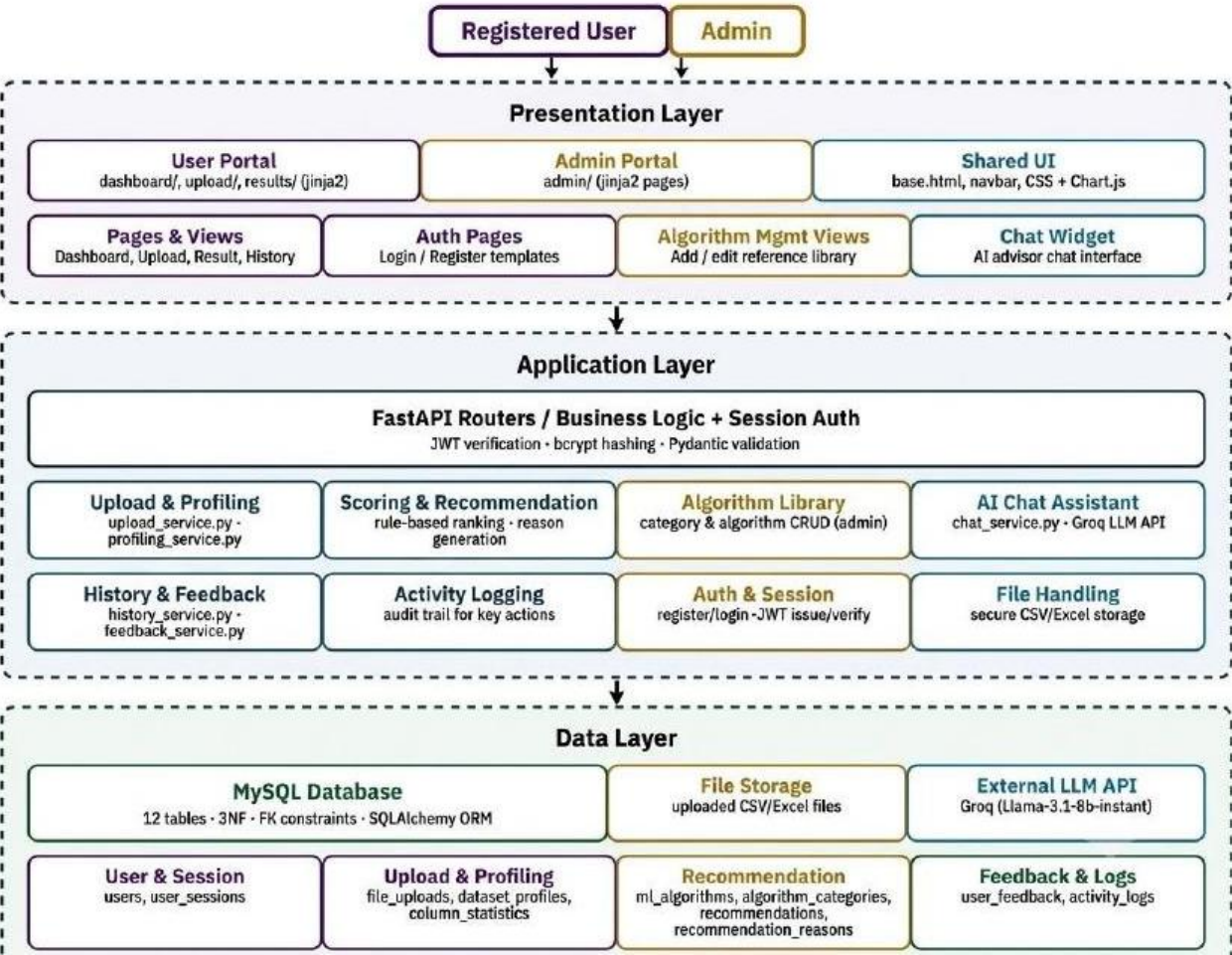
Auxiliary AI chat assistant



Jinja2 + Uvicorn

Server-side templates & ASGI server

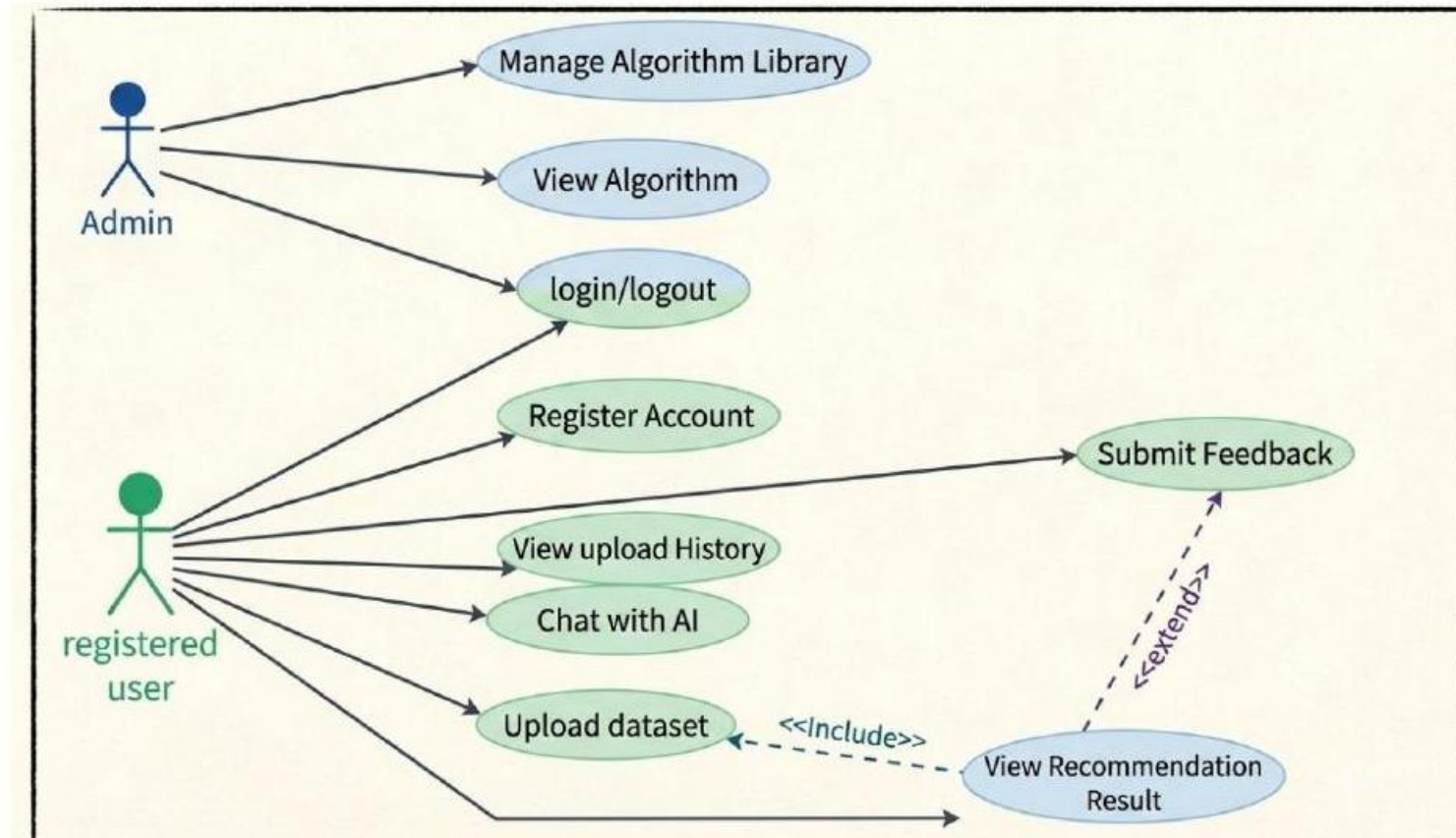
System Architecture



Three-Tier Design

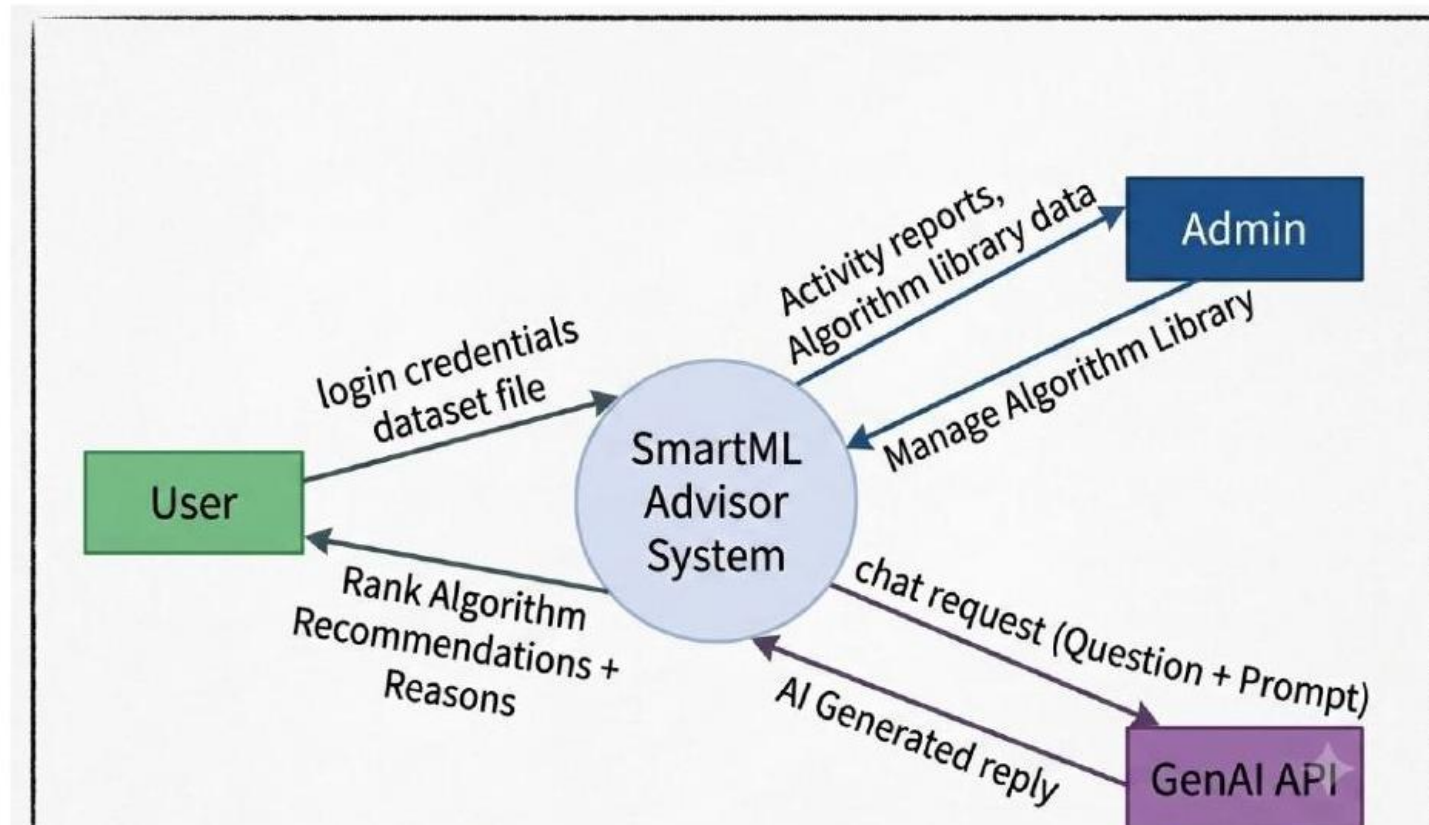
- Presentation: Browser + Jinja2 templates
- Application: FastAPI routers, services, JWT/bcrypt auth
- Data: MySQL (12 tables), file storage, external Groq LLM API

Use Case Diagram



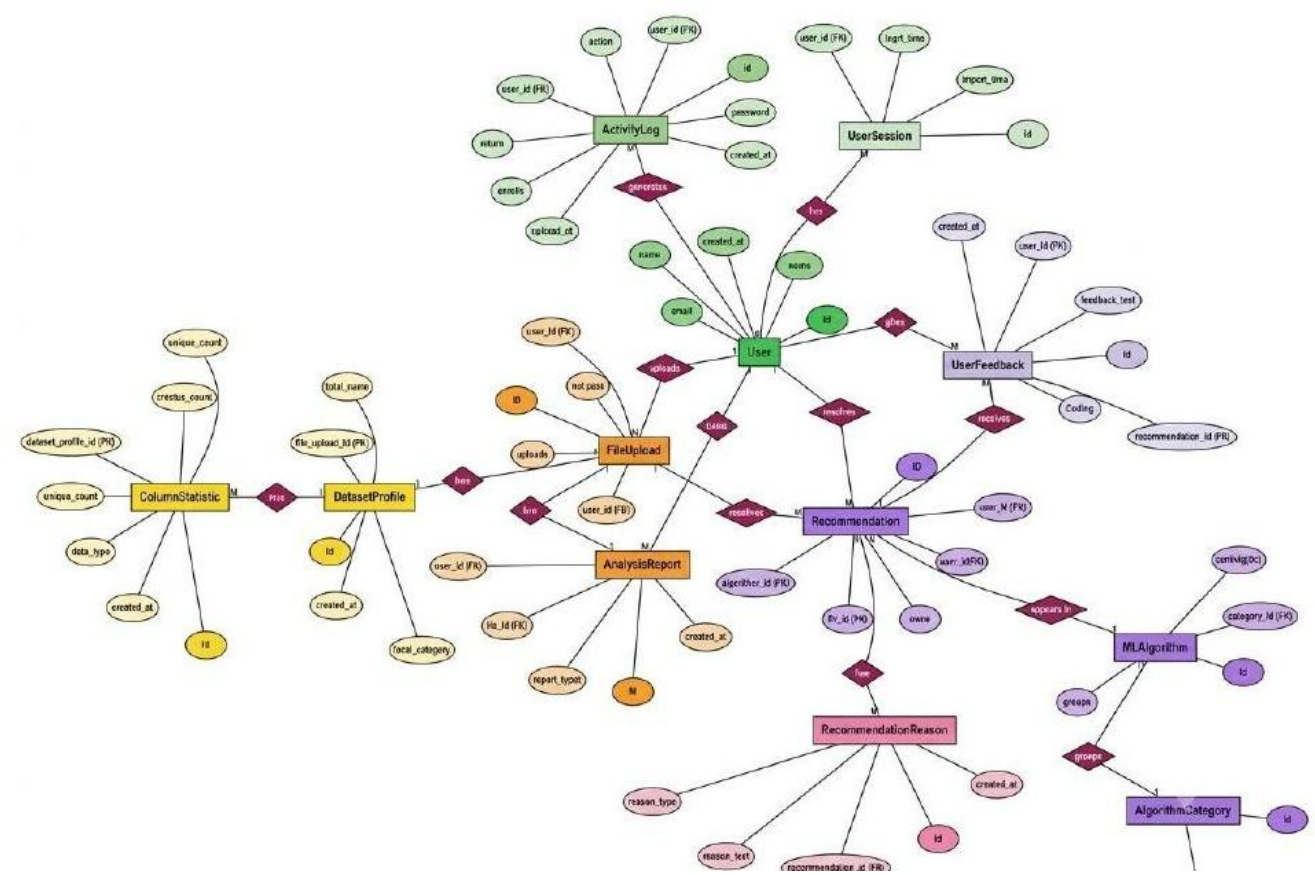
Registered User: register, login, upload, view results, browse library, history, feedback, chat | Admin: adds algorithm-library management

Context Diagram (Level-0 DFD)



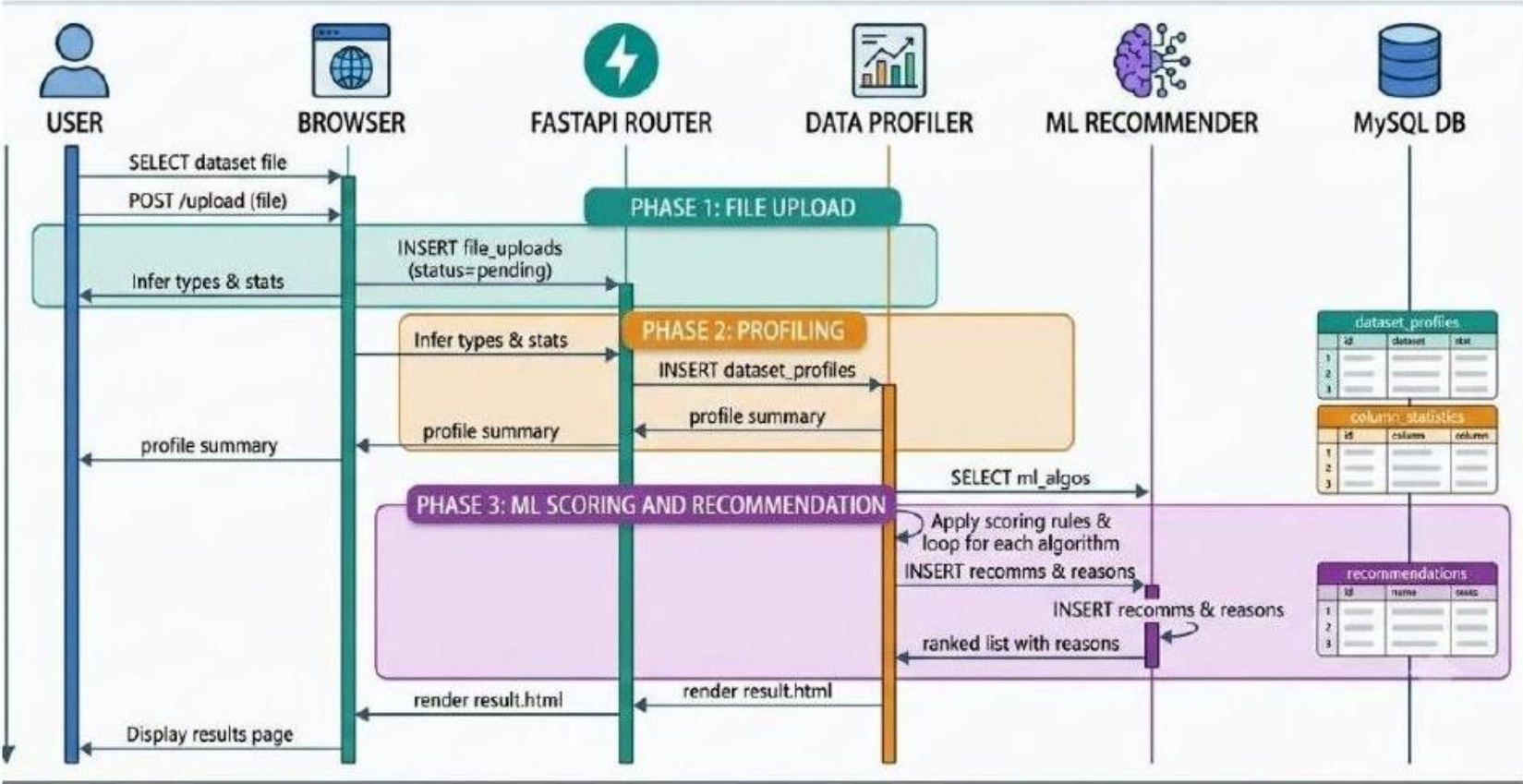
SmartML Advisor as one process exchanging data with the User, Admin, and the external GenAI API

Entity-Relationship Diagram



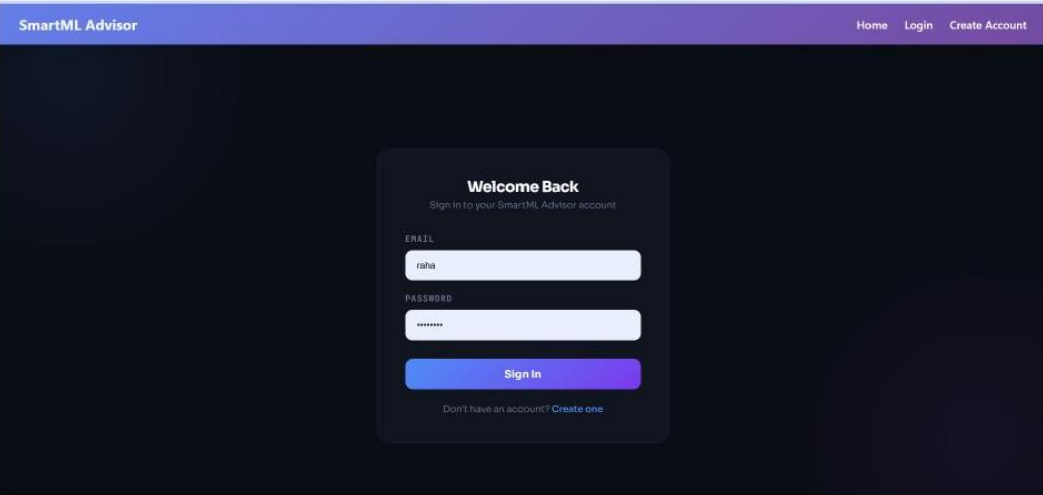
12 relational tables spanning identity, upload/profiling, recommendation, and feedback/auditing groups

UML Sequence Diagram



Upload → Profile → Score → Persist → Display, across Browser, FastAPI Router, Data Profiler, ML Recommender, and MySQL

Authentication Flow



Login Page

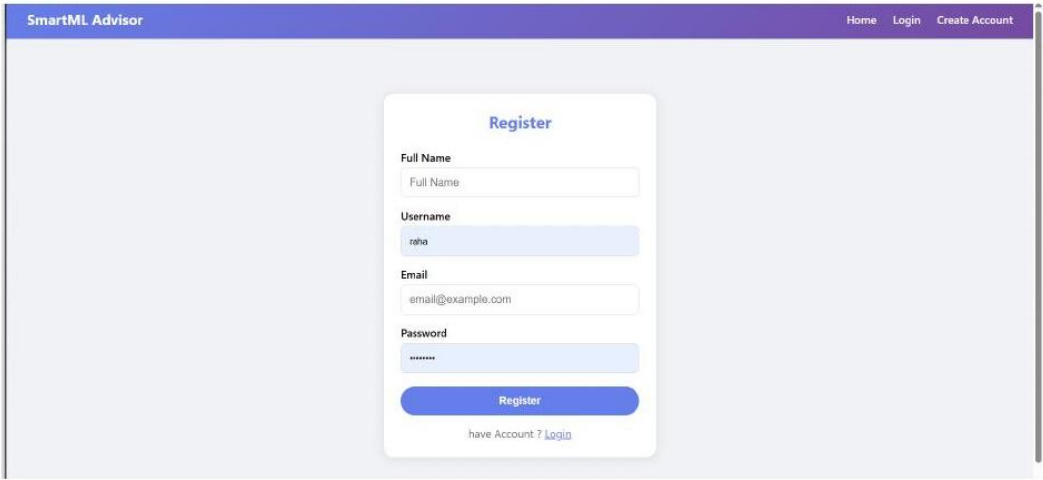
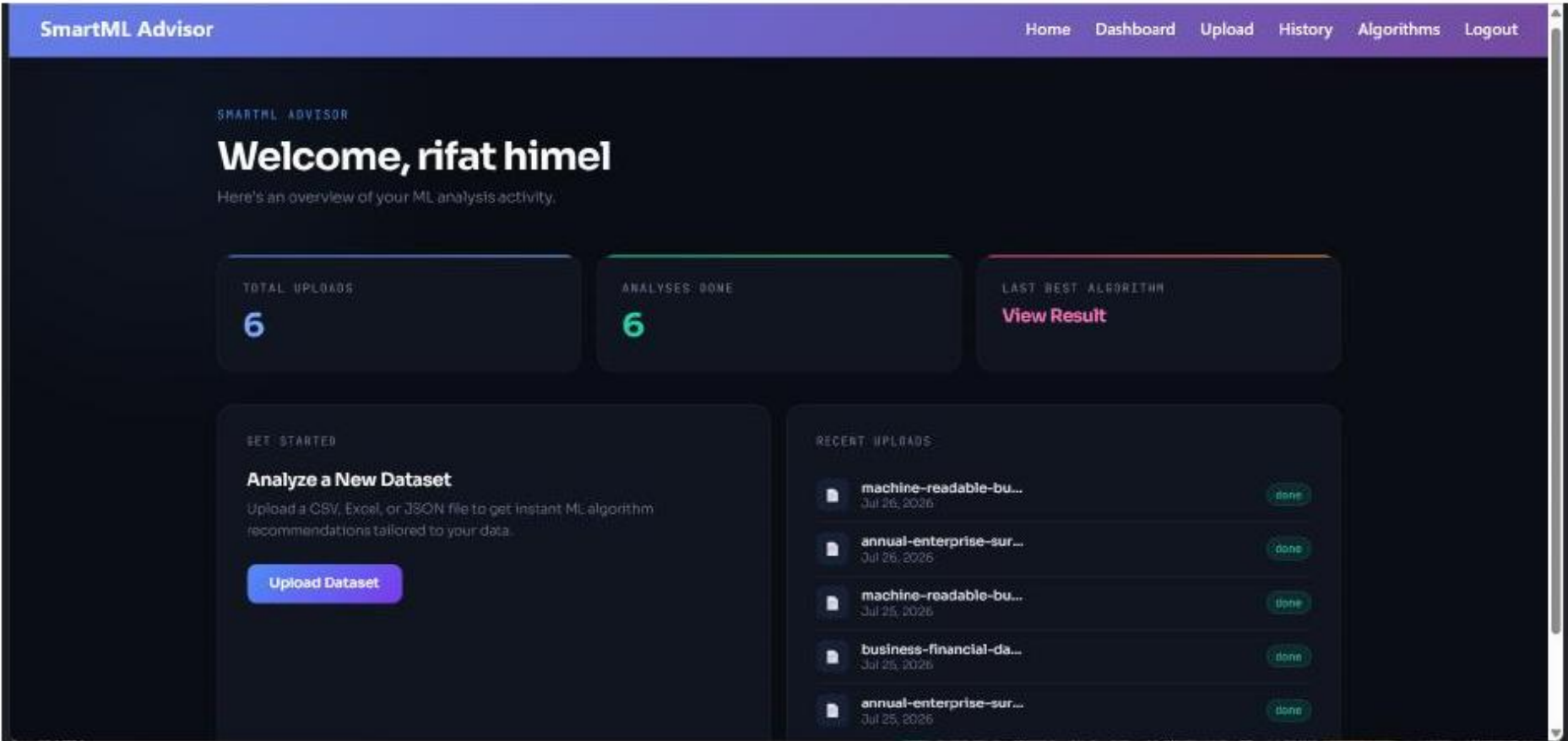


Figure 4.2: Registration Page

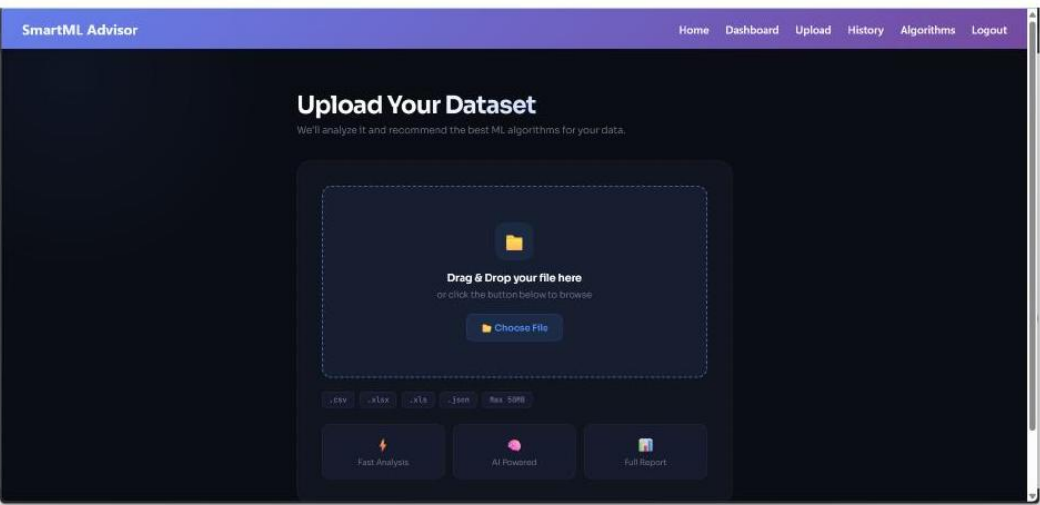
Registration Page

User Dashboard

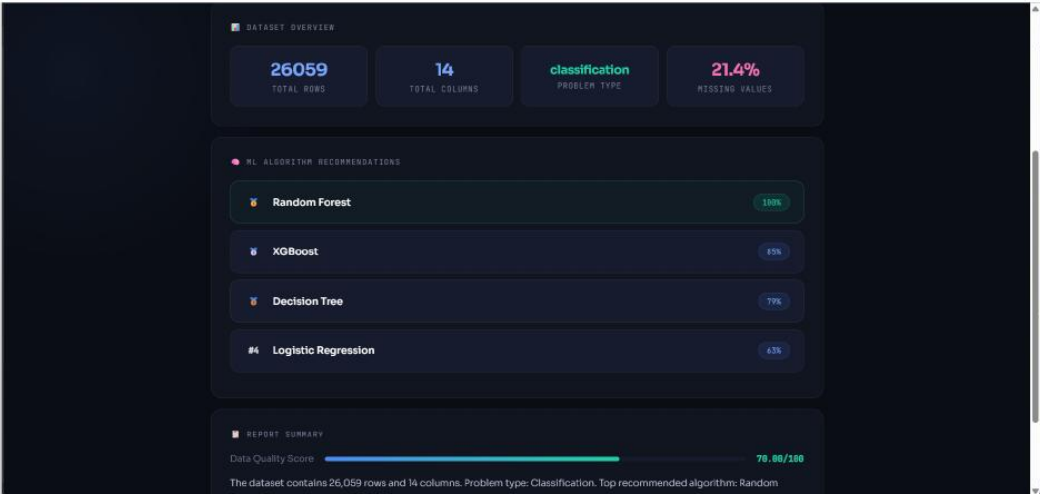


Central hub for uploads, recommendations, reference library, history & chat assistant

Upload → Recommendation Result

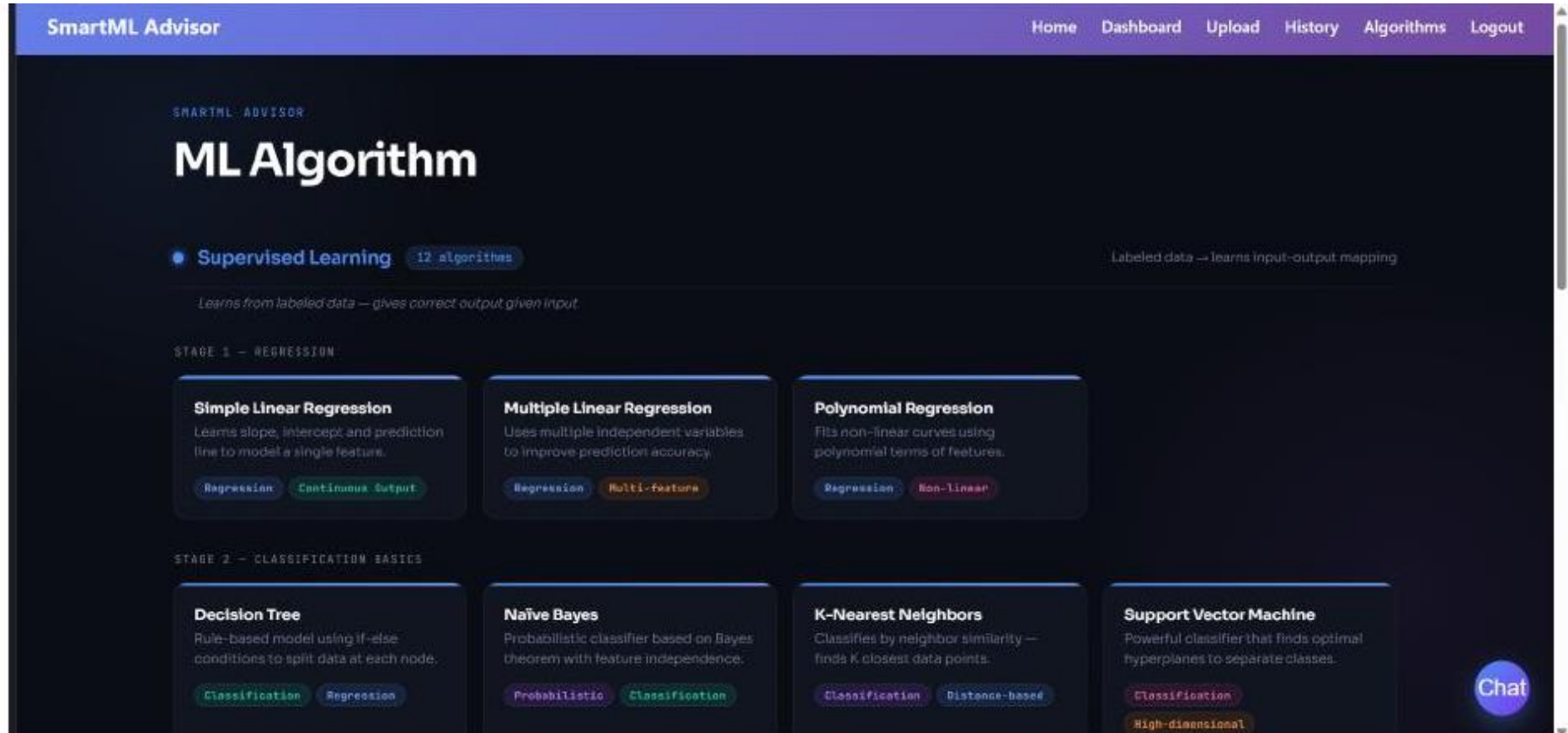


Dataset Upload



Ranked Algorithms with Suitability Scores

Algorithm Reference Library



Browsable by category (Classification, Regression, Clustering, Dim. Reduction, Association) — independent of any dataset upload

Database Design

Identity users, user_sessions
Algorithm Reference algorithm_categories, ml_algorithms
Upload & Profiling file_uploads, dataset_profiles, column_statistics
Recommendation recommendations, recommendation_reasons, analysis_reports
Feedback & Auditing user_feedback, activity_logs

12

Relational Tables
PK / FK constraints · ON DELETE CASCADE
/ SET NULL

Normalization Path: 1NF → 2NF → 3NF — full breakdown after the
CRUD/JOIN evidence.

CRUD + INNER / LEFT / RIGHT / simulated FULL OUTER / NATURAL JOINs
demonstrated on live data

Database Normalization

1NF

Atomic values

Boolean and status fields store a single atomic value instead of combined or multi-valued data.

2NF

No partial dependency

Every non-key attribute depends on the whole primary key, never on just part of a composite key.

3NF

No transitive dependency

Reference and category data is kept in its own table rather than duplicated through a non-key attribute.

Presenter cue: walk through 1NF → 2NF → 3NF as evidence the schema is structured, consistent and ready for reliable JOINS.

Database CRUD Operations — Read & Create

d	user_id	original_filename	stored_filename	file_type	file_size_bytes	file_path
1	2	dim.csv	56b40bfac3e64f18bfc65c55a8b8ba62_dim.csv	csv	28907	uploads\56b40bfac3e64f18bfc65c55a8b8ba62
2	2	weather_data.csv	6e7be7898fd14929b2f46bcb6e43a765_weather_data.csv	csv	257	uploads\6e7be7898fd14929b2f46bcb6e43a765
3	2	weather_data.csv	ba5c498b75e14c75b1af9b920b355a6e_weather_data.csv	csv	257	uploads\ba5c498b75e14c75b1af9b920b355a6e
4	2	weather_data.csv	2043e75c5820475c90b61115b95e45be_weather_data.csv	csv	257	uploads\2043e75c5820475c90b61115b95e45be
5	2	weather_data.csv	90b7272905cb4939b8101cbb589fe61f_weather_data.csv	csv	257	uploads\90b7272905cb4939b8101cbb589fe61f
6	2	Yield_df.csv	cef2c1e620aa41bc93279e83140686f2_Yield_df.csv	csv	1480286	uploads\cef2c1e620aa41bc93279e83140686f2
7	2	car1.csv	3fb967abe2d14186b2932de6b169f351_car1.csv	csv	22642	uploads\3fb967abe2d14186b2932de6b169f351
8	2	Yield_df.csv	1da3e4ed986d46058c796f7b7c446e59_Yield_df.csv	csv	1480286	uploads\1da3e4ed986d46058c796f7b7c446e59

READ — Initial file_uploads State

45	6	machine-readable-business-employment-data-mar-2026...	25a8b328d948409191f1be80c81e8138_machine-readable-...	csv	4044361	uploads\25a8b328d948409191f1be80c81e8138_machine-readable-business-employment-data-mar-2026...
46	6	annual-enterprise-survey-2025-financial-year-provi...	2a74b3a544de41aea600b3f8a7f5ad4f_annual-enterprise...	csv	2249137	uploads\2a74b3a544de41aea600b3f8a7f5ad4f_annual-enterprise-survey-2025-financial-year-provi...
47	6	machine-readable-business-employment-data-mar-2026...	6e89226228684ca995dc1bc3da23e17a_machine-readable-...	csv	4044361	uploads\6e89226228684ca995dc1bc3da23e17a_machine-readable-business-employment-data-mar-2026...
48	1	sales_data.csv	upload_002.csv	csv	204800	/uploads/upload_002.csv

CREATE — After Inserting New Upload

READ retrieves existing rows unchanged; CREATE inserts a new upload record with a new auto-incremented ID.

Database CRUD Operations — Update & Delete

id	user_id	original_filename	stored_filename	file_type	file_size_bytes	file_path	upload_status
1	2	dim.csv	56b40bfac3e64f18bfc65c55a8b8ba62_dim.csv	csv	28907	uploads\56b40bfac3e64f18bfc65c55a8b8ba62_dim.csv	completed
2	2	weather_data.csv	6e7be7899fd14929b2f46bcb6e43a765_weather_data.csv	csv	257	uploads\6e7be7899fd14929b2f46bcb6e43a765_weather_data.csv	completed
4	2	weather_data.csv	2043e75c5820475c90b61115b95e45be_weather_data.csv	csv	257	uploads\2043e75c5820475c90b61115b95e45be_weather_data.csv	completed

UPDATE — After Changing upload_status

id	user_id	original_filename	stored_filename	file_type	file_size_bytes	file_path
2	2	weather_data.csv	6e7be7899fd14929b2f46bcb6e43a765_weather_data.csv	csv	257	uploads\6e7be7899fd14929b2f46bcb6e43a765_weather_data.csv
4	2	weather_data.csv	2043e75c5820475c90b61115b95e45be_weather_data.csv	csv	257	uploads\2043e75c5820475c90b61115b95e45be_weather_data.csv
5	2	weather_data.csv	90b7272905cb4939b8101cbb589fe611f_weather_data.csv	csv	257	uploads\90b7272905cb4939b8101cbb589fe611f_weather_data.csv
6	2	yield_df.csv	cef2c1e620aa41bc33279e83140666f2_Yield_df.csv	csv	1480286	uploads\cef2c1e620aa41bc33279e83140666f2_Yield_df.csv
6	2	Yield_df.csv	cef2c1e620aa41bc33279e83140666f2_Yield_df.csv	csv	1480286	uploads\cef2c1e620aa41bc33279e83140666f2_Yield_df.csv
7	2	car1.csv	3fb967abe2d14186b2932de6b199f551_car1.csv	csv	22642	uploads\3fb967abe2d14186b2932de6b199f551_car1.csv
8	2	Yield_df.csv	1da3e4ed986d46058c796f7b7c446e59_Yield_df.csv	csv	1480286	uploads\1da3e4ed986d46058c796f7b7c446e59_Yield_df.csv
9	2	weather_data.csv	c56fb51c8a7c4d628f24f9afdbb19bf_weather_data.csv	csv	257	uploads\c56fb51c8a7c4d628f24f9afdbb19bf_weather_data.csv
10	2	Yield_df.csv	8a03159b2bc44a3089cd5ca52f7e78cf_Yield_df.csv	csv	1480286	uploads\8a03159b2bc44a3089cd5ca52f7e78cf_Yield_df.csv
11	2	pokemon.csv	60e8e26014ad4ad99a564d9762af1ec9_pokemon.csv	csv	160568	uploads\60e8e26014ad4ad99a564d9762af1ec9_pokemon.csv

DELETE — After Removing Test Upload

UPDATE modifies selected values while preserving row identity; DELETE removes the targeted test record.

SQL JOIN Operations — INNER & LEFT JOIN

original_filename	problem_type	suitability_score
weather_data.csv	classification	68.00
weather_data.csv	classification	68.00
weather_data.csv	classification	52.00
weather_data.csv	classification	52.00
students_records.csv	classification	78.00
students_records.csv	classification	62.00
students_records.csv	classification	58.00
students_records.csv	classification	42.00
movies.csv	regression	75.00

INNER JOIN — Uploads, Profiles & Recommendations

name	suitability_score	rank_position
Random Forest	68.00	1
Random Forest	78.00	1
Random Forest	68.00	1
Random Forest	83.00	1
Random Forest	83.00	1
Random Forest	83.00	2
Random Forest	91.00	1
Random Forest	68.00	1

LEFT JOIN — ml_algorithms & recommendations

INNER JOIN returns only matching rows across all tables; LEFT JOIN keeps every algorithm, even those never recommended.

SQL JOIN Operations — RIGHT & FULL OUTER JOIN

name	category
Random Forest	Classification
Logistic Regression	Classification
Decision Tree	Classification
XGBoost	Classification
K-Means	Clustering
Linear Regression	Regression
Ridge Regression	Regression

RIGHT JOIN — ml_algorithms & algorithm_categories

username	original_filename
ahsan	NULL
amin	NULL
emon	gt.csv
rafi	sales_data.csv
raha	dim.csv
raha	weather_data.csv
raha	Yield_df.csv
raha	car1.csv
raha	students_records.csv

FULL OUTER JOIN (simulated) — Users & Uploads

RIGHT JOIN preserves every category; MySQL lacks native FULL OUTER JOIN, so it is simulated via LEFT JOIN + RIGHT JOIN + UNION.

SQL JOIN Operations — NATURAL JOIN

upload_id	original_filename	problem_type
1	dim.csv	clustering
2	weather_data.csv	classification
4	weather_data.csv	classification
5	weather_data.csv	classification
6	Yield_df.csv	regression
7	car1.csv	clustering
8	Yield_df.csv	regression
9	weather_data.csv	classification
10	Yield_df.csv	regression

Matches views on the shared upload_id attribute, avoiding accidental matches on generic id columns.

Testing & Validation

18 / 18

Functional test cases passed

- Authentication & sessions
- Dataset upload & profiling
- Recommendation scoring & ranking
- History & feedback
- Chat assistant (Groq API)

Non-Functional Checks

- JWT-protected routes reject missing/invalid tokens
- Passwords verified via bcrypt (never stored in plain text)
- SQLAlchemy ORM parameterized operations prevent SQL injection
- Datasets sampled up to 50,000 rows for responsiveness
- Failed Groq API call doesn't crash the recommendation workflow
- Uploads/ownership scoped strictly per authenticated user

Results & Discussion

26,059

rows profiled
(weather_data.csv example)

100%

Random Forest suitability
(top-ranked, handles missing values)

9

real-world datasets tested
across all three problem types

- Classification datasets (weather_data.csv, students_records.csv) correctly favored Random Forest, Decision Tree, XGBoost.
- Regression datasets (Yield_df.csv, Real_Combine.csv) favored Linear/Ridge Regression and Random Forest Regressor.
- Datasets without a clear target (bal2.csv, car1.csv) were correctly routed to clustering algorithms (K-Means, DBSCAN).
- Algorithms unsuited to high-missing-value data (Logistic Regression, SVM) were consistently penalized — with stored, readable reasons.

Conclusion

- Achieved the objective of a database-centered, explainable ML algorithm recommendation platform.
- Connects secure authentication, profiling, rule-based scoring, reference library, and history/feedback through one shared MySQL database.
- Demonstrates core DBMS concepts: entity modeling, FK constraints with cascades, normalization to 3NF, all major JOIN types, ORM-based access.
- The scoring engine's most distinctive contribution: translating algorithm selection from a black-box decision into a structured, traceable, explainable relational design.

Limitations

- Fixed, hand-crafted scoring rules — not a learned meta-model.
- Problem-type detection uses simple heuristics (target column name / last column).
- Large datasets sampled to a maximum of 50,000 rows during profiling.
- Recommended algorithms are not actually trained/evaluated on the user's data — scoring is purely meta-feature based.
- AI chat assistant depends on a third-party hosted LLM (Groq API), adding external latency/availability risk.
- Testing is manual and scenario-based (18 cases); automated unit/security/performance tests are not yet comprehensive.

Future Work

- Optionally train top-ranked algorithms on a sample of the user's data and report real performance metrics (accuracy, F1, RMSE).
- Incorporate a trained meta-learning model using historical feedback to refine rule-based scores.
- Generate concrete preprocessing recommendations (imputation, encoding) from the stored dataset profile.
- Add streaming/chunked processing to support datasets beyond the 50,000-row limit.
- Build an administrative dashboard for algorithm-library management and activity monitoring.
- Add automated unit, integration, security, and performance test suites.

Thank You

Questions & Discussion

Abu Bakar Rafi · ID: 12208039 · Dept. of CSE, Comilla University